

KlariVision Perde Motorları

Dört algoritma, sıfırdan anlatım

YIN v1 · Pitch Engine V2 · VPM-like · HAPT v1
ve çevrimdışı referans pYIN

Her adımın matematiği, gerekçesi ve kod karşılığı
Yazılıma ve sinyal işlemeye yeni başlayanlar için

26 Ağustos 2026 · kod sürümü **37585a4**

İçindekiler

1. Ortak temeller — her motorun paylaştığı fikirler

Ses nedir, perde nedir · Çerçeveleme · RMS kapısı · Lag ve frekans · Sent · Üç aile: CMND, ACF, NSDF · Spektral prob

2. **YIN v1** — fark fonksiyonu ve kümülatif normalizasyon
3. **Pitch Engine V2** — dört tahminci, SWIPE' desteği, sabit gecikmeli Viterbi
4. **VPM-like** — normalize otokorelasyon, katlardan periyot rafinesi, spektral düzeltme
5. **HAPT v1** — NSDF, harmonikler-arası veto, faz kilitli frekans
6. **pYIN** — çevrimdışı olasılıksal referans
7. **Çevrimdışı iyileştirme katmanı** — `offline_track` profili
8. **Karşılaştırma** — hangi motor neyi iyi yapıyor, nerede yanılıyor
9. **Ek A** — parametre tabloları
0. **Ek B** — dosya haritası

1. Ortak temeller

Dört motor da aynı sorunu çözüyor ve aynı yapı taşlarını kullanıyor. Bu bölümü bir kez okursanız, sonraki dört bölüm sadece "bu motor bu taşları nasıl diziyo" sorusunun cevabı olur.

1.1 Ses nedir, perde nedir

Mikrofon size saniyede 44.100 (veya 22.050, 48.000) tane sayı verir. Her sayı, o andaki hava basıncıdır. Elimizde şöyle bir dizi olur:

$$x[0], x[1], x[2], \dots, x[N-1]$$

Klarnetten çıkan bir nota, bu dizide **kendini tekrar eden bir desendir**. Desen saniyede 440 kez tekrarlanıyorsa buna 440 Hz deriz ve "La" diye adlandırırız. Yani perde bulmak, şu tek soruyu cevaplamaktır:

Temel soru

Bu sayı dizisi kaç örnek sonra kendini tekrarlıyor?

O "kaç örnek" sayısına **lag** (gecikme) veya **periyot** denir; matematikte τ (tau) harfiyle gösterilir. Frekansa çevirmek tek bir bölme işlemidir:

$$f_0 = f_s / \tau$$

f_s : örnekleme hızı (Hz) · τ : periyot (örnek sayısı) · f_0 : temel frekans (Hz)

Örnek: 44.100 Hz'de $\tau = 100$ örnek ise $\rightarrow 44100 / 100 = 441$ Hz. Dört motorun tamamı, doğru τ 'yu bulmanın farklı yollarıdır. Farkları *hangi kanıta baktıkları ve ne zaman "bilmiyorum" dedikleridir*.

1.2 Çerçeveleme: sesi dilimlere bölmek

Perde zamanla değişir; tüm kaydı tek seferde analiz edemezsiniz. Ses kısa dilimlere (çerçeve / *frame* / pencere) bölünür ve her dilim için ayrı bir cevap üretilir.

- **Pencere boyu (window)**: bir dilimde kaç örnek var. Uzun pencere $\rightarrow$ frekans daha kesin ölçülür, ama hızlı değişimler (süsleme, vibrato tepesi) bulanıklaşır.
- **Adım (hop)**: bir sonraki dilime kaç örnek ilerlenir. Küçük adım $\rightarrow$ zamanda daha sık ölçüm; pencereler üst üste biner.

Üretimdeki değerler tüm motorlar için ortaktır: pencere 1536 örnek, adım 512 örnek. 44.1 kHz'de bu, 34,8 ms uzunluğunda bir pencere ve saniyede ~86 ölçüm demektir.

```
std::size_t window_size{1536};  
std::size_t hop_size{512};
```

core/include/klarivision/core/analysis_engine.hpp:35–36

Bir dilimin zaman damgası pencerenin **ortasıdır**, başı değil — ölçüm o pencerenin tamamını temsil eder:

```
const double time = (static_cast<double>(start) + config.window_size / 2.0) / rate;
```

core/src/analysis_engine.cpp:1190 – `PitchEngine::analyse_causal`

1.3 RMS kapısı: sessizlikte perde aramamak

Sessiz bir pencerede perde aramanın anlamı yoktur; gürültüde rastgele bir "periyot" bulunur. Bu yüzden her motor önce yüksekliğe bakar. RMS (kök ortalama kare) sinyalin enerji ölçüsüdür:

$$RMS = \sqrt{(1/N) \cdot \sum_{n=0}^{N-1} (x[n] - \bar{x})^2}$$

$\bar{x}$ ortalama (DC ofseti); önce çıkarılır, çünkü sabit bir kayma her otokorelasyonu şişirir

```
double centered_rms(std::span<const float> samples) {
    const double mean = std::accumulate(...) / samples.size(); // DC offset
    double sum = 0;
    for (const float sample : samples) { const double x = sample - mean; sum += x * x; }
    return std::sqrt(sum / samples.size());
}
```

core/src/analysis_engine.cpp:29–37

Üretim eşiği `minimum_rms = 0.015`. Bunun altındaki pencereler hiçbir motor tarafından analiz edilmez.

1.4 Sent: müzikal mesafe birimi

İki frekansın "ne kadar uzak" olduğunu Hz cinsinden ölçmek yanıltıcıdır: 100 Hz ile 110 Hz arası kulakta büyük, 1000 Hz ile 1010 Hz arası neredeyse duyulmaz bir farktır. Müzikte mesafe logaritmiktir. Birimi **senttir**; bir oktav 1200 senttir, bir yarım ses 100 senttir.

$$\text{cents}(a, b) = 1200 \cdot \log_2(a / b)$$

```
double cents_distance(double a, double b) { return std::abs(1200.0 * std::log2(a / b)); }
```

core/src/analysis_engine.cpp:66

Bu fonksiyon dört motorda da vardır ve şu üç yerde kullanılır: süreklilik cezası, oktav sıçraması tespiti, "aynı notaya döndük mü" kontrolü.

1.5 Üç akraba fonksiyon: CMND, ACF, NSDF

Motorların üçü periyodu zaman düzleminde arar ve hepsi aynı temel hamleyi yapar: **sinyali kendisinin τ kadar kaydırılmış hâliyle karşılaştırır**. Sadece karşılaştırma biçimleri farklıdır.

a) Ham otokorelasyon (ACF) — "ne kadar benziyor?"

$$r(\tau) = \sum_j x[j] \cdot x[j+\tau]$$

Sinyal kendine benzediğinde çarpımlar aynı işaretli olur ve toplam büyür. **Tepe** ararsınız. Sorunu: sonuç ses seviyesine bağlıdır, gür çalınca büyür. Bu yüzden hiçbir motor ham ACF kullanmaz, normalize eder.

b) YIN'in fark fonksiyonu ve CMND — "ne kadar farklı?"

$$d(\tau) = \sum_j (x[j] - x[j+\tau])^2$$

Farkın karesi. Doğru periyotta sıfıra yaklaşır; **dip** ararsınız. Genlikten görece bağımsızdır. YIN v1 ve V2'nin YIN üretici bunu kullanır (Bölüm 2).

c) NSDF (McLeod) — "iki uçtan normalize edilmiş benzerlik"

$$n(\tau) = 2 \cdot \sum_j x[j] \cdot x[j+\tau] / \sum_j (x[j]^2 + x[j+\tau]^2)$$

Pay ACF'nin kendisi, payda ise karşılaştırılan iki pencerenin toplam enerjisi. Sonuç $[-1, 1]$ aralığına oturur: 1,0 "mükemmel tekrar" demektir. Genlikten tamamen bağımsızdır ve doğrudan bir güven skoru gibi okunabilir. MPM ve HAPT bunu kullanır.

```
if (normalisation > 1e-12) {
    nsdf[lag] = 2.0 * correlation / normalisation; // McLeod's NSDF normalisation
}
```

core/src/mpm.cpp:74 · aynı core/src/hapt.cpp:110

Hepsinin ortak zaafı

Bir sinyal τ periyoduyla tekrarlıyorsa, 2τ , 3τ , 4τ ile de tekrarlar — iki tur da bir tekrardır. Bu yüzden her üç fonksiyon da doğru periyodun **katlarında** da güçlü sinyal verir. Motorların karmaşıklığının büyük kısmı bu tek belirsizliği çözmek içindir: buna **oktav hatası** (veya alt-harmonik / hayalet) denir.

1.6 Parabolik interpolasyon: tam sayıdan kurtulmak

τ örnek sayıdır, yani tam sayıdır. Ama gerçek periyot 100 değil 100,37 örnek olabilir. Bu fark müzikal olarak büyüktür: 44.1 kHz'de $\tau=100 \rightarrow 441,0$ Hz, $\tau=101 \rightarrow 436,6$ Hz; arada 17 sent var, kulakla duyulur.

Çözüm: tepenin (veya dibin) etrafındaki üç noktadan bir parabol geçirip **parabolün** tepe noktasını bulmak. Türev sıfırlanınca:

$$\delta = \frac{1}{2} \cdot (y_{\tau-1} - y_{\tau+1}) / (y_{\tau-1} - 2y_{\tau} + y_{\tau+1}), \tau^* = \tau + \delta$$

pay: dibin sola mı sağa mı kaçık olduğu · payda: eğrilik (ikinci türev)

Dört motor da bunu aynı biçimde, δ 'yi $\pm 0,5$ 'e kırparak uygular (düz bir eğride payda sıfıra yaklaşır ve δ patlayabilir):

```
const auto correction = std::abs(denominator) > 1e-9
    ? std::clamp(0.5 * (previous - next) / denominator, -0.5, 0.5)
    : 0.0;
const auto refined_lag = static_cast<double>(lag) + correction;
```

core/src/mpm.cpp:101–104 · eşleri: analysis_engine.cpp:124–127, vpm_like.cpp:40, hapt.cpp:33

1.7 Spektral prob: tek frekansta "orada ses var mı?"

Zaman düzlemi periyodu bulur ama *hangi harmoniğin gerçekten çaldığını* bilmez. Bunu öğrenmek için frekans düzlemine bakmak gerekir. Tam FFT yerine motorlar çoğu zaman **tek frekanslık DFT** kullanır: sinyali tam olarak sorulan frekanstaki bir sinüs ve kosinüsle çarpıp toplarlar.

$$X(f) = \sum_n w[n] \cdot x[n] \cdot \cos(2\pi f n / f_s) - i \cdot \sum_n w[n] \cdot x[n] \cdot \sin(2\pi f n / f_s)$$

$$w[n] = 0,5 - 0,5 \cdot \cos(2\pi n / (N-1)) \quad (\text{Hann penceresi})$$

$E(f) = |X(f)|^2$ enerji, $|X(f)|$ genlik, $\arg(X(f))$ faz

Neden tam FFT değil? Çünkü aday frekanslar (441,3 Hz gibi) FFT'nin bin merkezlerine denk gelmez; tek frekanslık DFT **tam istenen frekansta** ölçüm verir. Hann penceresi ise pencerenin kenarlarını yumuşatarak spektral sızıntıyı azaltır.

```
std::complex<double> hann_windowed_probe(std::span<const float> samples, double sample_rate, double
frequency_hz)
```

core/src/harmonic_probe.cpp:8–36 — ortak yardımcı; her motorun kendi kopyası da var

Bu probun **faz** bilgisi yalnız HAPT tarafından kullanılır (Bölüm 5); diğerleri sadece genliğe/enerjiye bakar.

Pencerenin çözünürlük sınırı

Bir pencere ne kadar kısaysa, spektrumda o kadar yakın iki frekansı ayırt edemez. Hann penceresinin ana lobu 4 DFT bini genişliğindedir:

$$\text{yarı genişlik} = 2 \cdot f_s / N$$

```
return 2.0 * sample_rate / static_cast<double>(sample_count);
```

core/src/harmonic_probe.cpp:45

1536 örnek @44,1 kHz için bu ≈ 57 Hz. HAPT bu sayıyı doğrudan bir karar olarak kullanır: "bu frekans için doğrudan spektral prob güvenilir mi, yoksa zaman düzlemine mi düşeyim?" (Bölüm 5.3).

1.8 Klarnetin fiziği: neden bu iş zor

Bu belgedeki karmaşıklığın büyük kısmı tek bir fiziksel gerçekten doğuyor.

Klarnet kapalı bir borudur

Bir ucu kamışla kapalı, silindirik bir boru yalnızca **tek sayılı harmonikleri** güçlü üretir: f , $3f$, $5f$... Çift harmonikler ($2f$, $4f$) fizik gereği zayıftır. Üstelik 3. harmonik çoğu zaman temel frekanstan **daha gürdür**.

Sonuç: otokorelasyon ailesinin tamamı, temel frekansı zayıf gördüğü için sürekli olarak yanlış katlara kilitlenmeye eğilimlidir. Klarnet ayrıca oktavdan ($2\times$) değil **onikiliden ($3\times$) aşırı üfler**, yani register değişimi de $3\times$ oranındadır.

Bu yüzden dört motorda da aynı üç savunma tekrar tekrar karşınıza çıkacak:

1. **Tek harmonik doluluğu**: gerçek bir temel frekans f , $3f$ ve $5f$ 'te enerji bırakır; sahte bir alt-harmonik bırakmaz.
2. **Aşağı sıçrama histerezisi**: ani ve büyük bir aşağı sıçrama, birkaç kare boyunca onaylanmadan yayınlanmaz.
3. **Bırakış (release) tespiti**: notanın bitişi, müzikal bir kısılmadan farklıdır; RMS çok daha hızlı düşer.

2. YIN v1

Projedeki en eski motor. Adındaki "v1" geriye dönük uyumluluk içindir, kalite sıralaması değil. Fikri tek cümleyle: **sinyali kendisiyle karşılaştırıp farkın en küçük olduğu gecikmeyi bul, ama farkı adil kıyaslanabilir hâle getirmek için normalize et.**

Nerede	Dosya	Ne işe yarıyor
Referans sürüm	<code>src/klarivision/pitch/yin.py</code>	Saf NumPy, 108 satır. Algoritmanın ders kitabı hâli.
Üretim (C++)	<code>core/src/analysis_engine.cpp:86</code>	<code>yin_candidates</code> + karar katmanı. JSON çıktılarıdaki <code>yin_v1</code> budur.
Canlı (Swift)	<code>LivePitchAnalyzer.swift:3772</code>	vDSP hızlandırılmalı kopya, aynı formül.

2.1 Fark fonksiyonu $d(\tau)$

Fikir basit: sinyali τ kadar kaydırılmış hâliyle karşılaştır. τ gerçek periyotsa kaydırılmış hâli orijinaline çok benzer, fark ≈ 0 olur.

$$d(\tau) = \sum_{j=0}^{W-1} (x[j] - x[j+\tau])^2$$

de Cheveigné & Kawahara (2002), denklem 6

Python referansında harfi harfine böyle yazılmış:

```
difference = np.array(
    [np.sum((frame[:-lag] - frame[lag:]) ** 2) for lag in lags],
    dtype=np.float64,
)
```

`src/klarivision/pitch/yin.py:76–79`

Neden fark, neden otokorelasyon değil?

Ham otokorelasyon (Bölüm 1.5a) ses seviyesine bağlıdır: gür çalınca büyür. YIN farkın karesini alır ve minimum arar — genlikten görece bağımsız, sıfıra yaklaşan temiz bir gösterge olur.

Üretimdeki hızlandırma: aynı formül, farklı hesap

Parantezi cebirsel olarak açarsanız:

$$d(\tau) = \sum_j x[j]^2 + \sum_j x[j+\tau]^2 - 2 \cdot \sum_j x[j] \cdot x[j+\tau]$$

ön enerji + kaydırılmış enerji – 2 × korelasyon

İlk iki terim, önceden hesaplanan **kümülatif enerji tablosundan** $O(1)$ sürede okunur; üçüncü terim tek bir nokta çarpımıdır ve Apple'ın vektör donanımında (`vDSP_dotpr`) koşar.

```
std::vector<float> energy(samples.size() + 1, 0);
for (std::size_t index = 0; index < samples.size(); ++index) {
    energy[index + 1] = energy[index] + samples[index] * samples[index];
}
...
const float correlation = float_dot(samples.data(), samples.data() + lag, count);
difference[lag] = std::max(0.0F,
    energy[count] + (energy[samples.size()] - energy[lag]) - 2.0F * correlation);
```

`core/src/analysis_engine.cpp:91–106`

`std::max(0.0F, ...)` şunun içindir: matematiksel olarak $d(\tau)$ negatif olamaz, ama float aritmetiğinde iki büyük sayının farkı -0.0000001 çıkabilir; kırılır.

Swift'teki karşılığı satır satır aynıdır (`LivePitchAnalyzer.swift:3794–3804`); yorumu neden böyle yapıldığını da söylüyor: eski Double/ArraySlice sürümü canlı yakalamayı ~9 Hz'e düşürüyormuş.

Aranan τ aralığı

$$\tau_{\min} = f_s / f_{\max} , \tau_{\max} = \min(f_s / f_{\min} , W/2)$$

W/2 sınırı önemli: pencerenin yarısından uzun bir periyodu ölçmek için yeterli tekrar yoktur.

Dün değişti: alt sınır 80 → 120 Hz

Üretim `minimum_frequency_hz` değeri 120 Hz'e çekildi. Bu bir görüntüleme tercihi değil: doğrudan $\tau_{\max}$ 'ı, yani otokorelasyonun arayabileceği en uzun periyodu kısıtlıyor. Fazla geniş bir lag aralığı, ACF'nin gerçek periyodun katına kilitlenmesine izin veren şeyin ta kendisi. Şükrü Tunar kaydında dinleyicinin işaretlediği her aşağı oktav hatası 83–160 Hz arasına düşmüş, motorların hemfikir olduğu hiçbir kare 146 Hz'in altına inmemiş. 120 Hz, sol klarnetin en pes sesini (~123,5 Hz) açıkta bırakıyor.

core/include/klarivision/core/analysis_engine.hpp:16–32

2.2 CMND: kümülatif ortalama normalize edilmiş fark

Ham $d(\tau)$ tek başına kullanılamaz, iki tuzağı var:

1. $d(0) = 0$ 'dır (sinyal kendisiyle birebir aynıdır). En küçüğü körü körüne ararsanız hep 0'ı bulursunuz.
2. τ büyüdükçe $d(\tau)$ genelde büyür; bu karşılaştırmayı adaletsiz kılar.

YIN'in çözümü: her $d(\tau)$ 'yu, o noktaya kadarki **ortalama** d değerine böl.

$$d'(\tau) = d(\tau) / [(1/\tau) \cdot \sum_{j=1}^{\tau} d(j)] = d(\tau) \cdot \tau / \sum_{j=1}^{\tau} d(j)$$

denklem 8; $d'(0) = 1$ tanımlanır

Payda bir ortalama değildir, dolayısıyla d' birimsizdir ve tipik değeri 1 civarındadır. Gerçek periyotta $d(\tau)$ ortalamanın çok altına düştüğü için d' **sıfıra çakılır**. Artık τ 'lar adil kıyaslanabilir ve 0,15 / 0,50 gibi mutlak eşikler anlam kazanır.

```
total += difference[lag];
if (total > 0) cumulative[lag] = difference[lag] * static_cast<float>(lag) / total;
```

core/src/analysis_engine.cpp:112–116 · Python: yin.py:80

Üretimde ders kitabından bir sapma

C++ ve Swift'te `difference` dizisi yalnız $[\tau_{\min}, \tau_{\max}]$ aralığında doldurulur, altındaki girdiler 0 kalır. Ama τ çarpanı hâlâ 1'den sayar. Yani payda gerçekte $\sum_{j=\tau_{\min}}^{\tau} d(j)$ iken çarpan τ 'dur; bu, ortalama $\tau/(\tau - \tau_{\min} + 1)$ katı küçültür ve d' olduğundan **büyük** çıkar.

Etkisi en küçük τ 'larda serttir: tam $\tau = \tau_{\min}$ noktasında $d' = \tau_{\min}$ olur (44,1 kHz / 1500 Hz için 29) — eşik 0,50'nin çok üstünde, yani en tiz birkaç lag pratikte hiç seçilemez. τ büyüdükçe çarpan 1'e yaklaşır ve fark kaybolur. Python referansı tüm lag'leri doldurduğu için bu sapmayı taşımaz.

2.3 Aday seçimi

Python referansı: mutlak eşik (klasik YIN)

Eşiğin altına düşen **ilk** τ alınır — en küçüğü değil, ilki. Sebebi kritiktir: gerçek periyot τ ise 2τ ve 3τ da dip yapar. En derin dibi seçerseniz oktav aşağı kayarsınız; ilk dibi seçmek doğru olanı verir.

```
below_threshold = np.flatnonzero(candidate_values < settings.threshold)
if len(below_threshold):
    candidate_index = int(below_threshold[0] + minimum_lag - 1)
    while (candidate_index < len(cumulative_mean) - 1
          and cumulative_mean[candidate_index + 1] <= cumulative_mean[candidate_index]):
        candidate_index += 1
else:
    candidate_index = int(np.argmin(candidate_values) + minimum_lag - 1)
```

src/klarivision/pitch/yin.py:82–93

`while` döngüsü şunu yapar: eşiği geçtiğiniz nokta dibin yamacı olabilir, iniş sürdükçe kaydırıp dibin tabanına inersiniz.

Üretim: tüm yerel minimumları topla

C++/Swift eşiği 0,50'ye gevşetir ve **tek bir cevap seçmez** — en fazla 12 aday döndürür, kararı üst katmana bırakır.

```
if (!(cumulative[lag] <= cumulative[lag - 1] &&
    cumulative[lag] < cumulative[lag + 1] &&
    cumulative[lag] < 0.50)) continue;
```

core/src/analysis_engine.cpp:122

Üç koşul birlikte "yerel minimum" tanıdır. Zayıf minimumların da tutulma sebebi Swift yorumunda açık: süreklilik puanlayıcısı, gür harmoniği yerine sessiz *temel frekans* kurtarabilsin diye.

2.4 Frekans ve güven

$$f_0 = f_s / \tau^* , \text{ güven} = \text{clip}(1 - d'(\tau), 0, 1)$$

Sezgi doğrudan: d' ne kadar sıfıra yakınsa sinyal o kadar temiz periyodiktir. $d' = 0,05 \rightarrow$ güven 0,95 (kristal netliğinde nota). $d' = 0,45 \rightarrow$ güven 0,55 (şüpheli; gürültü ya da iki nota arası geçiş).

Klasik YIN burada biter. Kalan her şey, bu ham tahmini klarnette kullanılabilir kılan `yin_v1` 'e özel karar katmanıdır.

2.5 Oktav kurtarma — ve dün eklenen tek-harmonik koşulu

Her aday için $2\times$ frekansına bakılır. Oradaki spektral enerji 8 kattan fazlaysa, gerçek temel bir oktav yukarıda olabilir; o frekans neredeyse tam güvenle listeye eklenir.

Bu pas tek başına 13 oktav hatasının 12'sini ürettiyordu

Ham $2\times$ baskınlığı, **gerçek temel frekansları** yukarı terfi ettiriyordu. Sebebi Bölüm 1.8: kapalı silindirden çift kısımlar yayılır, dolayısıyla gerçek bir temel frekans da "baskılanmış" görünebilir. Ayırt edici test **tek harmonik doluluğudur**: gerçek bir temel f , $3f$ ve $5f$ 'te enerji bırakır; f 'te hayalet olan (gerçek nota $2f$ olan) bir aday için $3f$ ve $5f$, gerçek notanın kısımları arasında boşluğa denk gelir.

```
const auto odd_support = [&](const double frequency) {
    double total = 0;
    for (const double multiple : {3.0, 5.0}) {
        const double probe = frequency * multiple;
        if (probe < rate / 2) total += spectral_energy(samples, rate, probe);
    }
    return total;
};
...
if (spectral_energy(samples, rate, upper) <=
    spectral_energy(samples, rate, result[index].frequency) * 8) continue; // 2x baskın mı?
if (odd_support(upper) <= odd_support(result[index].frequency)) continue; // tek harmonikler de üst hatta mı?
result.push_back({upper, 0.99999, std::log(0.99999)});
```

core/src/analysis_engine.cpp:148–171

2.6 Hayalet alt-harmonik cezası

Tersi durum: YIN gerçekte var olmayan bir alt-periyoda kilitlenebilir. Adayın kendi frekansında spektrumda neredeyse hiç enerji yokken $2\times/3\times$ katı baskınsa, bu bir **otokorelasyon hayaletidir**.

$$\text{ceza} = 0,15 \cdot \text{clip}((r - 6) / 20, 0, 1)$$

r = en baskın katın genliği / adayın kendi genliği

```
const double severity = std::clamp((existence.dominant_multiple_ratio - 6.0) / 20.0, 0.0, 1.0);
return 0.15 * severity;
```

core/src/analysis_engine.cpp:185–191 · eşik 6,0: core/src/harmonic_arbitration.cpp:70–77

6,0 eşiği ölçümle seçilmiş: zayıf ama gerçek bir klarnet temel frekansı kendi üçüncü harmonisinin ~3 katı gerisinde kalabiliyor; gerçek bir hayalet ise en belirsiz karesinde bile ~8 katına, temiz sürdürülen hâlinde 24–27 katına ulaşıyor. Eşik iki kümenin arasına, gerçek-ama-zayıf tarafa yakın konmuş.

Ceza **kademelidir, sert ret değildir** — kod yorumu gerekçeyi söylüyor: sürekliliği ve gerçekten belirsiz adayları adil biçimde yenmeli, ezip geçmemeli.

2.7 Süreklilik puanlaması

Müzik sıçramaz. Bir önceki karede 440 Hz duyduysanız, bu karede 880 Hz'den çok 442 Hz olması muhtemeldir.

$$\text{skor} = \text{güven} - \text{hayalet_cezası} - 0,30 \cdot \min(\text{sent_uzaklık} / 700, 1)$$

```
if (candidate.confidence < .55) continue;           // kendi başına çok zayıf
double score = adjusted_score(candidate);
if (previous) {
    const double distance = cents_distance(candidate.frequency, previous->frequency);
    score -= .30 * std::min(distance / 700.0, 1.0);
}
```

core/src/analysis_engine.cpp:249–255

Önceki kare yoksa (nota başlangıcı, artikülasyon boşluğu sonrası) süreklilik dayanağı da yoktur; orada baraj 0,76'ya yükselir ([analysis_engine.cpp:240](#)). Canlı Swift'in basit yolu da aynı 0,76'yı kullanır.

Seçilen aday yine de hayalet olarak işaretliyse, hiçbir şey yayınlanmaz. Gerekçe kodda yazılı: bir hayaletin temel frekans olarak yayınlanmaya *periyot düzleminde hiçbir hakkı yoktur*; sessiz kalmak, boşluk köprülemesinin kareyi çevresinden kurtarmasına izin verir.

2.8 Aşağı sıçrama histerezisi

Son yayınlanan perdenin 900 sentten fazla **altına** düşen bir aday tek karede kabul edilmez; 3 ardışık kare onaylamalıdır (her biri bekleyen adayın 360 senti içinde).

```
if (impl->pending_yin_confirmations < 3) return {}; // henüz yeterince onaylanmadı
```

core/src/analysis_engine.cpp:565

2.9 Bırakış tespiti ve yayın kapısı

Bir notanın bitişi müzikal bir *decrescendo*'dan farklıdır: RMS çok daha hızlı düşer. İki ardışık %20'lik düşüş bırakış tetikler ve süreklilik durumu sıfırlanır.

```
if (impl->previous_yin_rms > 0 && rms < impl->previous_yin_rms * .80) ++impl->yin_release_falls;
...
if (impl->yin_release_falls >= 2) impl->yin_release_active = true;
```

core/src/analysis_engine.cpp:462–470

İki ayrı RMS tepesi tutulur: hızlı sönen ($\times 0,85$, yavaş-fade kapısı için) ve yavaş sönen ($\times 0,995$, toparlanma eşiği için).

Son kapı: güveni 0,90'ın altında **ve** RMS'i kendi son tepesinin %35'inin altında olan bir aday yayınlanmaz — "sönen reverb kuyruğu, gerçek nota değil".

```
if (choice && !impl->yin_release_active && !(choice->confidence < .90 &&
    rms < impl->recent_yin_rms_peak * .35)) {
    publication = EngineFrame{source_time, choice->frequency, choice->confidence};
}
```

core/src/analysis_engine.cpp:582–584

2.10 Boşluk köprüleme

Kısa kopmalar (en fazla 7 kare) son bilinen perde tekrarlanarak doldurulur — ama **sadece** boşluğun iki ucu da yeterince güvenliyse

ve birbirine 90 sentten yakınsa. Yani köprü "aynı notaya geri döndük" kanıtıyla açılır.

```
std::max(last->confidence, frame->confidence) >= minimum_endpoint_confidence &&  
std::min(last->confidence, frame->confidence) >= weaker_floor &&  
cents_distance(*last->frequency_hz, *frame->frequency_hz) <= 90
```

core/src/analysis_engine.cpp:301-305 – [append_bridged](#), yin_v1 ve hapt_v1 ortak

[yin_v1](#) için güven tabanı 0,55'tir. 30 kare boyunca hiç yayın olmazsa süreklilik durumu tamamen sıfırlanır.

2.11 Bir karenin tam yolculuğu

1536 örnek	
↓ RMS < 0,015 ? → sessizlik, çık	analysis_engine.cpp:29
↓ DC çıkar	
↓ $d(\tau) = \sum (x[j] - x[j+\tau])^2$ her $\tau \in [\tau_{min}, \tau_{max}]$	analysis_engine.cpp:102
↓ $d'(\tau) = d(\tau) \cdot \tau / \sum d(j)$	analysis_engine.cpp:116
↓ yerel minimumlar, $d' < 0,50$ (≤ 12 aday)	analysis_engine.cpp:122
↓ parabolik düzeltme δ	analysis_engine.cpp:124
↓ $f_0 = f_s / (\tau + \delta)$, güven = $1 - d'(\tau)$	analysis_engine.cpp:130
↓ oktav kurtarma ($2 \times$ enerji $8 \times$ + tek harmonik doluluğu)	analysis_engine.cpp:148
↓ hayalet cezası – süreklilik cezası	analysis_engine.cpp:185, 253
↓ aşağı sıçrama: 3 kare onay	analysis_engine.cpp:558
↓ bırakış / yavaş-fade kapısı	analysis_engine.cpp:582
↓ boşluk köprüleme (≤ 7 kare, ≤ 90 sent)	analysis_engine.cpp:274
EngineFrame{ zaman, frekans, güven }	

Elle takip etmek isterseniz

En iyi başlangıç [src/klarivision/pitch/yin.py](#) – 108 satır, dallanma yok. `_estimate_frame` fonksiyonuna tek bir pencere verip ara dizileri yazdırarak $d(\tau)$ ve $d'(\tau)$ eğrilerini gözle görebilirsiniz.

3. Pitch Engine V2

*YIN v1'in temel varsayımı şudur: "bir tahminci çalıştır, kararı hemen ver." V2 bu varsayımın ikisini de reddeder. **Dört bağımsız tahminci** çalıştırır, hiçbirinin kararını erken kabul etmez, ve son kararı **beş kare geriden**, geleceğe bakarak verir.*

3.1 Temel fikir: kanıtı erken çökertme

YIN v1'de her karede tek bir cevap seçilir ve o cevap sonraki karenin süreklilik dayanağı olur. Bir kare yanlış giderse, hata sürekliliğe sinip yayılabilir.

V2 bunun yerine her karede bir **aday listesi** tutar ve her adayın nereden geldiğini (`CandidateSource`) hatırlar:

```
enum class CandidateSource { yin, autocorrelation, mpm, spectral };
```

core/include/klarivision/core/pitch_engine_v2.hpp:11–16 – "V2 keeps this provenance instead of collapsing every observation into one early YIN decision"

Karar, adaylar arasından değil, **kare dizisi üzerinden bir yol** olarak verilir (3.5).

3.2 Dört aday kaynağı

a) YIN üretici

Bölüm 2'deki CMND algoritmasının aynısı. Farkı: periyodikliği 0,50'nin altındaki adaylar baştan atılır.

```
candidates.erase(std::remove_if(candidates.begin(), candidates.end(), [](const auto& item) {
    return item.periodicity < 0.50;
}), candidates.end());
```

core/src/pitch_engine_v2_session.cpp:311–314

b) Normalize otokorelasyon

Klasik ACF, ama enerjiyle normalize edilmiş ve **tek** bir aday döndürür. Seçim stratejisi Bölüm 4'teki VPM-like ile aynıdır: en güçlü tepeyi değil, en güçlüye "yeterince yakın" olan **en kısa lag**'i seç — böylece oktav-aşağı hayaletlerden kaçınılır.

core/src/pitch_engine_v2_session.cpp:146–208 – `autocorrelation_candidate`

c) MPM (McLeod Pitch Method) – NSDF

Bölüm 1.5c'deki NSDF. Her yerel maksimum bir adaydır ve hepsi döndürülür (en fazla 12).

$$n(\tau) = 2 \cdot \sum x[j] \cdot x[j+\tau] / \sum (x[j]^2 + x[j+\tau]^2)$$

core/src/mpm.cpp:9–133 (referans) · core/src/pitch_engine_v2_session.cpp:215 (vDSP'li eş sürüm)

d) Spektral oktav adayı

Mevcut adayların 2x frekansında enerji 8 kat baskınsa, o frekans `spectral` kaynaklı yeni bir aday olarak eklenir — mevcut adayın üzerine yazılmaz, yanına konur.

```
if (spectral_energy(samples, sample_rate, upper) >
    spectral_energy(samples, sample_rate, candidates[index].frequency_hz * 8) {
    candidates.push_back(PitchCandidate{
        upper, std::max(0.92, candidates[index].periodicity), 0, CandidateSource::spectral,
    });
}
```

core/src/pitch_engine_v2_session.cpp:344–350

3.3 SWIPE' harmonik desteği

Her aday, tam FFT üzerinden hesaplanan **SWIPE' benzeri spektral çekirdek** ile sınanır. Bu, V2'nin diğer motorlarda olmayan en özgün parçasıdır.

Fikir: gerçek bir temel frekans f ise, spektrumda f , $2f$, $3f$... noktalarında **tepeler**, aralarında **çukurlar** olmalıdır. SWIPE' bunu bir "iç çarpım" olarak ölçer:

$$\text{skor}(f) = \sum_k w_k \cdot [S(kf) - \frac{1}{2}(S((k-\frac{1}{2})f) + S((k+\frac{1}{2})f))] / \sqrt{(\sum w_k^2 \cdot \sum S(kf)^2)}$$
$$w_k = 1/\sqrt{k}$$

$S(\cdot)$: Hann pencere FFT'nin karekök-genlik spektrumu, binler arası doğrusal interpolasyonla okunur

Üç tasarım kararı önemli:

- **Tepe eksi komşu çukurlar.** Yalnız tepe yüksekliğine bakmak, gürültülü/yassı bir spektrum bölgesini de yüksek puanlardı. Fark almak, dar ve gerçek bir harmonik tepesini ödüllendirir.
- **Sadece 1. ve asal harmonikler pozitif kanıt sayılır.** Sebep: asal olmayan bir harmonik ($4f$, $6f$, $9f$...) aynı zamanda *başka*, daha pes bir temel frekansın da harmoniğidir; asal olanlar bu karışıklığa çok daha az düşer.
- **Payda tüm harmonikleri sayar.** Böylece daha gür veya daha kalabalık bir spektrum otomatik olarak yüksek puan almaz — skor bir kosinüs benzerliği gibi $[0, 1]$ aralığında kalır.

```
if (harmonic != 1 && !is_prime(harmonic)) {
    continue; // only the fundamental (harmonic 1) and prime harmonics contribute positive evidence
}
const auto weight = 1.0 / std::sqrt(static_cast<double>(harmonic));
...
inner_product += weight * (peak - 0.5 * (left_valley + right_valley));
```

core/src/swipe_prime.cpp:142–151 · asallık testi: swipe_prime.cpp:15–25

Bütün adaylar tek bir FFT çağrısında toplu olarak puanlanır (`swipe_prime_harmonic_supports`); FFT, dosya içinde yazılmış radix-2 Cooley–Tukey uygulamasıdır (`swipe_prime.cpp:44–100`).

3.4 Aday puanı: üç kanıtın harmanı

Her aday üç bağımsız kanıtla puanlanır.

$$\text{emisyon} = 0,40 \cdot \text{periyodiklik} + 0,20 \cdot \text{uzlaşma} + 0,40 \cdot \text{SWIPE}'_{\text{desteği}}$$

$\text{uzlaşma} = 1,0$ (başka bir tahminci 55 sent içinde aynı frekansı söylediyse), aksi hâlde $0,25$

Uzlaşma (agreement) terimi V2'nin kalbidir: birbirinden bağımsız iki tahmincinin neredeyse aynı frekansı söylemesi, tek bir tahmincinin yüksek özgüveninden daha güçlü kanıttır.

```
for (std::size_t other = 0; other < candidates.size(); ++other) {
    if (candidates[other].source != candidates[index].source &&
        cents(candidates[other].frequency_hz, candidates[index].frequency_hz) <= 55) {
        agrees = true; break;
    }
}
const double agreement = agrees ? 1.0 : 0.25;
double score = 0.40 * candidates[index].periodicity + 0.20 * agreement + 0.40 * supports[index];
```

core/src/pitch_engine_v2_session.cpp:370–380

Spektral kaynaklı aday için puan $0,90$ 'a **tabanlanır**: o aday zaten yalnızca $8\times$ doğrudan spektral kanıtla var olduğu için, periyot düzlemindeki tahmincilerin alt harmonikte anlaşması onu ezmemelidir.

Yüksek register barajı

900 Hz'in üstündeki bir adaya güvenmek için ya tahminciler arası uzlaşma, ya da kendi başına yüksek periyodiklik **ve** spektral destek gerekir:

```
return candidate.agreement_support || (candidate.frequency_hz > 900 &&
    candidate.periodicity >= 0.80 && candidate.harmonic_support >= 0.75);
```

core/src/pitch_engine_v2_session.cpp:399–400

3.5 Sabit gecikmeli Viterbi izleyici

Burası V2'yi diğer üç motordan gerçekten ayıran yerdir. Diğerleri **tamamen nedenseldir** (causal): her kareyi görür görmez karar verirler. V2 kararı **5 kare geciktirir** ve o beş karenin kanıtını da hesaba katarak geriye dönük en iyi yolu seçer.

Viterbi nedir?

Elinizde her kare için birkaç aday var. Bir "yol", her kareden bir aday seçmektir. Her yolun puanı iki parçadan oluşur:

$$\text{yol_puanı} = \sum_{\text{kare}} \text{emisyon}(\text{seçilen aday}) + \sum_{\text{kare}} \text{geçiş}(\text{önceki aday} \rightarrow \text{bu aday})$$

Emisyon "bu aday kendi başına ne kadar inandırıcı" (3.4), **geçiş** ise "bu sıçrama müzikal olarak ne kadar makul" demektir. Viterbi, tüm olası yollar arasından en yüksek puanlıyı, kombinatoryal patlama olmadan, dinamik programlamayla bulur.

$$\text{geçiş}(a \rightarrow b) = -0,18 \cdot \min(|\text{cents}(a,b)| / 700, 1) - 0,12 \text{ [eğer } |\text{cents}| \geq 850]$$

ikinci terim: bir karelik oktav/onikili sıçramasına ek ücret

```
auto penalty = 0.18 * std::min(cents / std::max(1.0, width_cents), 1.0);
if (cents >= 850.0) {
    penalty += 0.12; // extra surcharge for a near-octave-or-larger jump
}
return -penalty;
```

core/src/fixed_lag_tracker.cpp:25-33

Ek ücretin kasten **ölçülü** tutulduğunu kod yorumu söylüyor: sürdürülen bir register değişimi, sonraki beş karenin emisyon kanıtıyla yine de kazanabilmeli.

Neden "sabit gecikmeli"?

Tam Viterbi tüm kaydı görmeyi gerektirir; canlı çalarken bu imkânsızdır. V2 uzlaşmayı şurada bulur: yalnız **5 kare** ileri bakar. 512 örneklik adımda 48 kHz'de bu ≈ 53 ms sabit gecikme demektir — algılanabilir sınırın hemen altında.

```
if (frames_.size() <= lag_frames_) {
    return std::nullopt; // henüz yeterli ileri-bakış yok
}
return resolve_front(); // tampon dolu: en eski kareyi şimdi karara bağla
```

core/src/fixed_lag_tracker.cpp:51-57

Önemli bir ayrıntı: **sessizlik yolu keser**. Dinamik program yalnız en öndeki kareden başlayan kesintisiz seslendirilmiş bloğu kullanır; araya giren bir sessizlik müzikal sürekliliği kopardığı için ileri-bakış orada durur.

```
std::size_t voiced_count = 0;
while (voiced_count < frames_.size() && !frames_[voiced_count].candidates.empty()) {
    ++voiced_count;
}
```

core/src/fixed_lag_tracker.cpp:77-80

Yol çözüldükten sonra geri-ışaretçiler (back-pointers) geriye doğru yürünür ve **yalnız en eski karenin** kararı yayınlanır; kalan dört kare tamponda kalıp bir sonraki turda daha çok ileri-bakışla yeniden değerlendirilir.

3.6 Yayın kapıları

Viterbi bir aday seçtikten sonra bile yayın garantisi değildir. Sırayla:

Kapı	Kural
RMS kapısı	Kare sessizse yayın yok.
Bırakış (release)	Kare bırakış anında yakalandıysa yayın yok. Karar 5 kare geciktiği için, kararın ait olduğu karenin bırakış durumu ayrı kuyruklarda saklanır — geç çözüm asla başka bir pencerenin kapısını uygulamaz.
Yüksek register	900 Hz üstü için 3.4'teki ek kanıt barajı.
Güven barajı	0,70 — <i>ama</i> son yayınlanan perdenin 90 senti içindeki bir kare 0,55'e düşebilir ("stable weak contour"): sürekliliğin kendisi de kanıttır.
Harmonik histerezis	Aşağı harmonik sıçraması için onay gecikmesi (<code>harmonic_filter</code>).

```
const bool stable_weak_contour = last_published_ && tentative->confidence >= 0.55 &&
    cents(tentative->frequency_hz, last_published->frequency_hz) <= 90;
if (tentative->confidence >= 0.70 || stable_weak_contour) {
    publication = harmonic_filter(*tentative);
}
```

core/src/pitch_engine_v2_session.cpp:490–495

Başlangıç köprüsü (`bridge_startup`)

Zarif bir ayrıntı: bir notanın atağı çoğu zaman biraz belirsiz başlar, periyodiklik ancak birkaç kare sonra oturur. Bu kareler atılırsa notanın başlangıç zamanı geç görünür. V2 bunun yerine ilk gerçek yayına kadar olan zayıf kareleri **tamponlar**; gerçek yayın gelip ayn perdeye yakın çıkarsa, tamponlanan koşu geriye dönük olarak serbest bırakılır.

core/src/pitch_engine_v2_session.cpp:579–612

3.7 Bir karenin tam yolculuğu

1536 örnek	
↓ RMS kapısı + bırakış tespiti	v2_session.cpp:423–460
↓ YIN adayları (periyodiklik $\geq 0,50$)	v2_session.cpp:62
↓ + ACF adayı (tek)	v2_session.cpp:146
↓ + MPM/NSDF adayları (≤ 12)	v2_session.cpp:215
↓ + spektral oktav adayı ($2\times$ enerji $8\times$)	v2_session.cpp:344
↓ SWIPE' desteği (tek toplu FFT)	swipe_prime.cpp:167
↓ emisyon = $0,40 \cdot \text{per} + 0,20 \cdot \text{uzlaşma} + 0,40 \cdot \text{SWIPE}'$	v2_session.cpp:379
↓ — 5 kare tampon —	
↓ Viterbi: emisyon + geçiş cezası, en iyi yol	fixed_lag_tracker.cpp:83–125
↓ yayın kapıları ($\text{RMS} \cdot \text{bırakış} \cdot 900 \text{ Hz} \cdot 0,70/0,55$)	v2_session.cpp:479–499
↓ başlangıç köprüsü + boşluk köprüleme	v2_session.cpp:579, 613
PublishedPitchFrame{ zaman, frekans, güven }	

V2'nin ödünleşimi

Kazandığı: tek karelik hatalar yola oturmaz; dört bağımsız kanıt birbirini denetler; 900 Hz üstünde disiplinli.

Ödediği: ~53 ms sabit gecikme, ve kare başına belirgin biçimde daha çok hesap (dört tahminci + FFT). Çevrimdışı iyileştirme katmanında V2'ye daha gevşek bir geçiş genişliği verilmesinin sebebi de budur: zaten kendi Viterbi'si var (Bölüm 7).

4. VPM-like

Tadao Yamaoka'nın Vocal Pitch Monitor için anlattığı yöntemin bağımsız uygulaması. Uygulamanın kendi eşikleri yayınlanmadığı için bu bir kopya değil, "VPM-benzeri" bir yeniden inşadır. Üç aşaması var: **normalize otokorelasyon, katlardan periyot rafinesi, spektrumla harmonik düzeltme.**

core/include/klarivision/core/vpm_like.hpp:10–15 · core/src/vpm_like.cpp

4.1 Normalize otokorelasyon (ACF)

Ham ACF ses seviyesine bağlıdır (Bölüm 1.5a). VPM-like bunu, karşılaştırılan iki alt pencerenin enerjilerinin **geometrik ortalamasına** bölerek çözer:

$$r(\tau) = \sum_j x[j] \cdot x[j+\tau] / \sqrt{E_{\text{ön}} \cdot E_{\text{gecikmeli}}}$$
$$E_{\text{ön}} = \sum_{j=0}^{N-\tau} x[j]^2, \quad E_{\text{gecikmeli}} = \sum_{j=\tau}^N x[j]^2$$

sonuç $\approx 1,0 \rightarrow$ dalga biçimi τ örnek sonra kendini tekrarlıyor

İki enerji terimi de önceden hesaplanan bir **ön-ek toplam tablosundan** $O(1)$ sürede okunur — YIN'in kullandığı hilenin aynısı.

```
const auto leading = energy[count]; // [0, count) alt penceresinin enerjisi
const auto delayed = energy[centered.size()] - energy[lag]; // [lag, size) alt penceresinin enerjisi
const auto normalization = std::sqrt(std::max(1e-18, leading * delayed));
double dot = 0.0;
for (int index = 0; index < count; ++index) dot += centered[index] * centered[index + lag];
correlation[lag] = dot / normalization;
```

core/src/vpm_like.cpp:234–241

Tepe seçimi: en güçlüyü değil, en kısıyı

Burada VPM-like'in karakteristik hamlesi geliyor. En yüksek ACF tepesini almak, oktav-aşağı hayaletine kilitlenmenin en kolay yoludur (çünkü 2τ da neredeyse aynı kadar güçlüdür). Bunun yerine:

$$\text{ Kabul_eşiği } = \max(0,38, r(\tau_{\text{en güçlü}}) \cdot 0,90)$$

bu eşiği geçen en kısa lag seçilir — yani en yüksek frekans

```
const auto acceptance = std::max(
    config.minimum_periodicity,
    correlation[strongest] * config.near_strongest_ratio
);
auto selected = strongest;
for (const auto peak : peaks) {
    if (correlation[peak] >= acceptance) { selected = peak; break; }
}
```

core/src/vpm_like.cpp:277–286

Tepeler artan lag sırasında bulunduğu için, döngüde eşiği geçen **ilk** tepe otomatik olarak en kısa lag'dir. Basit ama etkili: "neredeyse eşit güçteyse, daha tiz olanı seç."

4.2 Katlardan periyot rafinesi

ACF'nin lag kuantizasyonu (tam sayı örnek sınırı) en çok tiz seslerde göze batar. Parabolik interpolasyon (Bölüm 1.6) yardım eder ama yeterli değildir. Yamaoka'nın çözümü zarif:

Fikir

ACF'nin 2τ , 3τ , 4τ ... civarında da tepeleri vardır. Bu tepelerin her biri, **katına bölündüğünde**, aynı τ periyodunun *bağımsız bir yeniden ölçümüdür*. Üstelik daha uzun bir aralık üzerinden ölçüldükleri için tek-örnek kuantizasyon hatasına daha az duyarlıdır. Hepsinin ağırlıklı ortalamasını almak, tek bir tepedeki parabolik interpolasyondan daha keskin bir sonuç verir.

$$\tau_{\text{refine}} = \sum_{m=1}^6 (\tau_m^* / m) \cdot w_m / \sum_m w_m, \quad w_m = r(\tau_m) \cdot m$$

τ_m^* : m- τ civarında bulunan tepenin parabolik olarak rafine edilmiş konumu

Ağırlık hem tepenin gücüne hem de katın büyüklüğüne orantılıdır: daha yüksek kat = daha uzun ölçüm aralığı = daha güvenilir.

```
for (int multiple = 2; multiple <= config.maximum_period_multiple; ++multiple) {
    const auto expected = selected * multiple;
    const auto radius = std::max(2, selected / 6);          // kayma payı
    ...
    if (correlation[local] < config.minimum_periodicity * 0.8) continue; // inandırıcı tepe yok
    const auto weight = correlation[local] * static_cast<double>(multiple);
    weighted_period += parabolic_peak(correlation, local) / static_cast<double>(multiple) * weight;
    weight_sum += weight;
}
const auto refined_period = weighted_period / std::max(1e-12, weight_sum);
```

core/src/vpm_like.cpp:319-341

4.3 Spektrumla harmonik düzeltme

ACF *periyodikliği* bulur ama hangi harmoniğin gerçekten temel olduğunu bilmez. Bu aşama ACF'nin kararını gerçek spektrumla sınar: adayın kendisi ve 1/3x, 1/2x, 2x, 3x akrabaları tek tek ölçülür ve **en pes makul olanı** seçilir.

Neden en pes? Çünkü asıl hata modu "eksik/zayıf temel frekans, güçlü harmonikler"dir — tersi çok daha nadirdir.

Bir aday "makul" sayılmak için üç testi birden geçmeli:

Test	Kural	Eşik
Yerel spektral tepe	Genliği, her iki yanındaki komşu ölçümlerden açıkça büyük olmalı	$\geq \%103$
Mutlak destek	Genliği mutlak olarak yeterince gür olmalı	$\geq 0,005$
Görelî destek	ACF'nin kendi frekansındaki genliğin makul bir kesri olmalı	$\geq \%8$

```
const auto side_offset = std::max(sample_rate / samples.size(), frequency * 0.012);
const auto left = spectral_amplitude(samples, sample_rate, frequency - side_offset);
const auto right = spectral_amplitude(samples, sample_rate, frequency + side_offset);
const auto is_local_peak = amplitude >= left * 1.03 && amplitude >= right * 1.03;
const auto is_absolute = amplitude >= config.minimum_absolute_spectral_amplitude;
const auto is_relative = base_amplitude <= 1e-12 ||
    amplitude >= base_amplitude * config.minimum_relative_spectral_amplitude;
```

core/src/vpm_like.cpp:387-393

İki kritik kısıt

Bu düzeltme pası tek yönlüdür ve iki yerde kilitlenmiştir:

1. Yukarı terfi yasak

Parlak bir üst harmonik, güçlü bir ACF temel frekansını 2x/3x'e **terfi ettiremez**. Düzeltme yalnız aşağı yönlüdür (eksik temeli onarmak için). Bu, projedeki D-011 kararının gereğidir.

```
const auto does_not_promote = config.allow_spectral_promotion ||
    frequency <= autocorrelation_frequency * (1.0 + 1e-12);
```

2. ACF zaten en güçlüyü seçtiyse, aşağı düzeltme yok

Aşağı doğru spektral düzeltme, *daha erken ve neredeyse-güçlü* bir ACF tepesini onarmak için vardır. Seçilen tepe zaten global en güçlü tepeseyse, onu f/2 veya f/3 ile ezmek kısa alt-harmonik adacıkları üretiyordu (aleyhte holdout kaydında ölçüldü).

```
const auto does_not_override_strongest_acf =
    frequency >= autocorrelation_frequency * (1.0 - 1e-12) || selected != strongest;
```

core/src/vpm_like.cpp:398-406

Çıkış güveni, seçilen ACF tepesinin yüksekliğidir ve **0,80**'in altındaysa hiçbir şey yayınlanmaz — bu, dört motor içindeki en yüksek kare-içi güven barajıdır.

4.4 VPMLikeTracker: aşağı harmonik sıçraması muhafızı

Kare tahmincisi kasten **durumsuzdur** (stateless). Kareler arası hafıza ayrı bir sınıfta durur ve tek bir işi vardır: şüpheli bir **aşağı** sıçramayı geciktirmek. Sıradan perde hareketini asla yumuşatmaz.

"Şüpheli" tanımı iki koşulun birleşimidir:

$$|cents(yeni, eski)| \geq 650 \text{ ve } yeni/eski \text{ oranı} \in \{ 1/3, 1/2, 2/3 \} \pm 110 \text{ sent}$$

```
bool is_downward_harmonic_jump(const double previous, const double current) {
    if (current >= previous || std::abs(cents_between(current, previous)) < 650.0) return false;
    for (const auto ratio : {1.0 / 3.0, 0.5, 2.0 / 3.0}) {
        if (std::abs(cents_between(current / previous, ratio)) <= 110.0) return true;
    }
    return false;
}
```

core/src/vpm_like.cpp:23–33

Yani büyük ama "harmonik oranı"na benzemeyen bir düşüş (gerçek bir müzikal atlama) anında kabul edilir; sadece tam 1/2 veya 1/3 gibi oranlar beklemeye alınır. Bu ayrım motorun zekâsının özüdür.

Üç sonuçlu karar

Durum	Sonuç
Eski (üst) hat hâlâ en az 2,5x daha güc	Doğrudan veto. Eski perde yayınlanmaya devam eder, güven 0,55'e kırılır.
Rakip üst hat kanıtı hiç yok	Yeni pes tahmin anında kabul edilir (veto edilecek bir şey yok).
Üst hat hâlâ gerçek bir spektral tepe	3 ardışık kare onayı beklenir (her biri 180 sent içinde); onaya kadar eski perde tekrarlanır.

core/src/vpm_like.cpp:120–163

Kod yorumu bunun bir **veto** olduğunu, spektral terfi olmadığını özellikle vurguluyor: gerçek bir pes atak, eski üst hattı zaten kaybeder ve normal onay yolundan geçer — geleceğe bakmaya gerek kalmadan.

4.5 Oturum katmanı: bırakış, boşluk, atak bekleme

Üretimde VPM-like, `analysis_engine.cpp` içinde kendine ait, diğer motorlardan daha ayrıntılı bir durum makinesiyle sarılır. Ana durumlar:

Durum	Anlamı ve çıkış koşulu
<code>waiting_for_attack</code>	Bir bırakış veya süresi dolmuş boşluktan sonra: hem güvenli bir tahmin hem zarfın son tepesinin %40'ına tırmanması gerekir. İkisi birden olmadan çıkmaz.
<code>release_suspected</code>	Yalnız iki sonuç mümkündür: zarf <i>aynı</i> kontura döner (yanlış alarm → tamponlanan boşluk serbest bırakılır), veya <i>farklı</i> bir kontura döner (gerçek yeni atak → eski kontur temizlenir).
<code>weak_contour_hold</code>	Güvenli tahmin yok ama zayıf bir tahmin izlenen kontura katılıyor: gürültülü zayıf tahmini takip etmek yerine konturun kendi frekansı yeniden yayınlanır.

core/src/analysis_engine.cpp:600–920

Bu ayrıntı seviyesi tesadüf değil: VPM-like, dinleyici hakemliğinde **sessiz kalması gerektiğinde sessiz kalan** motor olarak ölçüldü. Çevrimdışı katmandaki "başıboş nokta" temizleyicisi bile eşliğini (0,80) doğrudan VPM-like'ın kendi yayın barajından alır (Bölüm 7.3).

4.6 Bir karenin tam yolculuğu

```
1536 örnek (double'a çevrilir)
↓ DC çıkar + RMS kapısı          vpm_like.cpp:184-201
↓ normalize ACF, her  $\tau \in [\tau_{min}, \tau_{max}]$  vpm_like.cpp:234
↓ yerel maksimumlar → tepe listesi vpm_like.cpp:246
↓ en güçlüye %90 yakın en kısa lag seçilir vpm_like.cpp:277
↓  $2\tau..6\tau$  tepelerinden ağırlıklı periyot rafinesi vpm_like.cpp:319
↓ spektral akrabalar: 1/3, 1/2, 1, 2, 3 × vpm_like.cpp:361
↓ en pes "makul" olan seçilir (3 test + 2 kısıt) vpm_like.cpp:387
↓ güven = ACF tepe yüksekliği, < 0,80 ise yayın yok vpm_like.cpp:459
↓ VPMLikeTracker: aşağı harmonik sıçraması muhafızı vpm_like.cpp:96
↓ oturma durum makinesi (atak / bırakış / boşluk) analysis_engine.cpp:600
EngineFrame{ zaman, frekans, güven }
```

5. HAPT v1

"Harmonik-Adaptif Faz-kilitli İzleyici". Diğer üçünün eşiti, dördüncü bağımsız motor. İki fikri kendine özgüdür: **çözünürlük-farkında harmonikler-arası veto** ve **faz kilitli anlık frekans rafinesi**. Birincisi klarnetin fiziğini doğrudan bir karar kuralına çeviriyor; ikincisi kareler arası hiçbir hafıza kullanmadan alt-sent doğruluk üretiyor.

core/include/klarivision/core/hapt.hpp:10–31 · core/src/hapt.cpp · docs/HAPTPitchEngine.md

5.1 Temel: NSDF

HAPT, McLeod'un NSDF'sini kullanır (Bölüm 1.5c) — VPM-like'ın ACF'si gibi normalize, ama iki uçlu:

$$n(\tau) = 2 \cdot \sum_j x[j] \cdot x[j+\tau] / \sum_j (x[j]^2 + x[j+\tau]^2)$$

```
table.values[lag] = 2.0 * correlation / normalisation; // McLeod's NSDF normalisation
```

core/src/hapt.cpp:110

Küçük ama düşünülmüş bir ayrıntı: tablonun iki ucuna birer sıfır dolgu konur, böylece **tam $\tau_{\min}$** 'deki bir tepe de tepe sayılabilir. Bunsuz, $f_{\max}$ 'a çok yakın gerçek bir temel frekans hiç aday olamazdı.

```
// One padding zero at each end ... lets the peak search treat the two lag
// boundaries like any interior point instead of silently excluding them.
table.values.assign(static_cast<std::size_t>(maximum_lag + 2), 0.0);
```

core/src/hapt.cpp:92–98

0,05'in üstündeki her yerel maksimum bir adaydır. Şimdi asıl iş başlıyor: bu adaylar arasından hangisinin *gerçek* temel frekans olduğunu seçmek.

5.2 Fikir 1: harmonikler-arası veto

NSDF, gerçek periyodu katlarından ayıramaz — 2τ , 3τ , $k\tau$ hepsi neredeyse aynı derecede periyodik görünür (Bölüm 1.5 kutusu). HAPT'in çözümü şunu sormaktır:

Sorulan soru

"Bu aday gerçekten bir oktav (veya bir onikili) **fazla pes** olsaydı, spektrumda **başka nerede** enerji olurdu? Orada enerji var mı?"

Aday frekansı f olsun. Eğer f gerçek temelse, enerji f , $2f$, $3f$... noktalarında olur. Ama gerçek temel $2f$ ise (yani f bir oktav fazla pesse), o zaman **f 'in tek katları arasındaki** ara noktalarda — yani "yarım ızgara" $(k-\frac{1}{2})\cdot f$ konumlarında — hiç enerji olmaması gerekir. Orada enerji varsa, aday f gerçekten f 'tir; yoksa aday muhtemelen $2f$ 'in hayaletidir.

$$\text{yarım_veto} = E_{\text{yarım ızgara}} / (E_{\text{yarım ızgara}} + E_{\text{kendi harmonikleri}})$$

üçte_bir_veto: aynı $(k-\frac{1}{3})\cdot f$ ve $(k-\frac{2}{3})\cdot f$ problemleri için

0 = veto yok; 1'e yakın = "bu aday muhtemelen bir kat fazla pes"

```
for (int k = 1; k <= harmonic_count; ++k) {
    harmonic_multiples.push_back(k);           // adayın kendi k. harmoniği
    half_multiples.push_back(k - 0.5);         // "yarım ızgara" probu
    third_multiples.push_back(k - 1.0 / 3.0);   // "üçte bir ızgara" problemleri
    third_multiples.push_back(k - 2.0 / 3.0);
}
```

core/src/hapt.cpp:241–246

Çözünürlük farkındalığı

Burada HAPT'ı özel kılan ikinci katman var. Yarım-ızgara probu, ancak pencere o frekans aralığını **çözebiliyorsa** anlamlıdır. Pencere kısaysa, prob komşu harmoniğin sızıntısını ölçer ve saçmalar. HAPT bunu Hann ana lobu genişliğinden (Bölüm 1.7) hesaplar:

$$\text{yarım ızgara tabanı} = 3 \cdot (2f_s/N) , \text{ üçte bir ızgara tabanı} = 4,5 \cdot (2f_s/N)$$

```
const auto half_width = hann_main_lobe_half_width_hz(samples.size(), sample_rate);
const auto half_grid_floor_hz = 3.0 * half_width;
const auto third_grid_floor_hz = 4.5 * half_width;
```

core/src/hapt.cpp:185–187

Aday bu tabanın **üstündeyse** doğrudan spektral prob kullanılır. **Altındaysa** — yani pencere o kadar ince ayırım yapamıyorsa — zaman düzlemine düşülür ve NSDF'nin katındaki değeri kıyaslanır:

$$\text{yarım_veto} = \text{clip}(n(2\tau) / n(\tau) - 1, 0, 1)$$

oktav-aşağı tepe kendi tepemizden ne kadar güçlüyse o kadar şüpheli

```
const auto lower = nsdf_at_lag(*table, static_cast<int>(std::lround(2.0 * refined_lag)));
if (lower >= 0.0 && periodicity > 1e-9) {
    half_veto = clamp01(lower / periodicity - 1.0);
}
```

core/src/hapt.cpp:262–265

Bu "elimdeki pencereyle hangi kanıtı ölçebilirim?" refleksi, diğer üç motorda yok.

Tek harmonik doluluğu — ve neden çift harmonikler serbest bırakılır

Klarnet kapalı silindirdir: tek kısımlar (1., 3., 5.) tonu taşır, çiftler fizik gereği zayıftır (Bölüm 1.8). HAPT bunu doğrudan bir ölçüye çevirir ve **yalnız {1, 3, 5}'e bakar** — çiftlere hiç kısıt koymaz.

$$\text{doluluk} = (\text{mevcut olan tek harmonik sayısı}) / (\text{ölçülebilen tek harmonik sayısı})$$

"mevcut" = genliği, en gür harmoniğin en az %10'u

Neden çiftlere bakılmıyor?

Çünkü tam da o, iki durumu birbirine karıştıran testtir. "Çift harmonikleri zayıf" olmak, klarnette hem **gerçek bir temel frekansın** hem de **periyodu ikiye katlanmış bir hayaletin** imzasıdır. Ayırt eden şey çiftlerin zayıflığı değil, **teklerin varlığıdır**.

Anlamlılık ağırlığı: iki geçişli puanlama

İnce bir tuzak var: gerçek hiçbir spektral içeriğe denk gelmeyen bir aday (örneğin gerçek tondan çok pes bir alt-harmonik) için 1., 3., 5. harmoniklerin hepsi sadece pencere sızıntısı gürültüsüdür — *hepsi birbirine benzer*, dolayısıyla doluluk oranı sahte biçimde "%100 dolu" çıkabilir.

HAPT bunu iki geçişle çözer. Birinci geçişte her adayın **mutlak** harmonik enerjisi de ölçülür; ikinci geçişte doluluk terimi, adayın karedeki en güçlü harmonik enerjisinden ne kadar pay aldığına göre ağırlıklandırılır:

$$\text{anlamlılık} = \text{clip}(E_{\text{aday}} / E_{\text{en güçlü aday}}, 0, 1)$$

$$\text{doluluk_çarpanı} = 0,35 + 0,65 \cdot \text{doluluk}$$

$$\text{etkin_çarpan} = 1 - \text{anlamlılık} \cdot (1 - \text{doluluk_çarpanı})$$

anlamlılık $\approx 0 \rightarrow$ çarpan 1'e (nötr) gider · anlamlılık $\approx 1 \rightarrow$ tam klarnet biçimli modülasyon

Nihai puan

$$\text{puan} = \text{clip}[\text{periyodiklik} \cdot \text{etkin_doluluk_çarpanı} \cdot (0,10 + 0,90 \cdot \text{anlamlılık}) \\ \cdot (1 - 0,9 \cdot \text{yarım_veto}) \cdot (1 - 0,9 \cdot \text{üçte_bir_veto}) + \text{süreklilik_bonusu}]$$

süreklilik bonusu: 700 sente kadar doğrusal azalarak +0,10'a kadar

```
const auto score = clamp01(
    candidate.periodicity * effective_occupancy_factor * significance_factor *
    (1.0 - 0.9 * candidate.half_veto) * (1.0 - 0.9 * candidate.third_veto) + continuity_bonus
);
```

core/src/hapt.cpp:367–370

Her veto puanı en fazla %90 kesebilir — yani hiçbirini tek başına mutlak ret değildir, hepsi çarpan olarak yarışır.

5.3 Fikir 2: faz kilitli anlık frekans

Parabolik interpolasyon (Bölüm 1.6) periyodu alt-örnek doğrulukla verir, ama bu hâlâ bir *tahmindir*. HAPT son bir rafine daha yapar ve bunun için **fazı** kullanır — bu bilgiyi kullanan tek motor odur.

Fikir

Aynı 1536 örneklik pencerenin içinden, 512 örnek kaydırılmış **iki alt pencere** (her biri 1024 örnek) alın. Aynı frekansta prob atın. Eğer prob frekansı gerçek frekansa tam eşitse, iki ölçümün fazı aynı olur. Fark varsa, faz kayması doğrudan frekans sapmasıyla orantılıdır.

$$\Delta\phi = \arg(X_{\text{geç}} \cdot \text{conj}(X_{\text{erken}}))$$

$$df = -\Delta\phi \cdot f_s / (2\pi \cdot 512)$$

±93,75 Hz belirsizliksiz aralık (docs/HAPTPitchEngine.md:73)

```
const auto phase_difference = std::arg(chosen_late * std::conj(chosen_early));
const auto deviation_hz = -phase_difference * sample_rate /
    (2.0 * std::numbers::pi * static_cast<double>(kIFShiftSamples));
const auto refined_probe_frequency = chosen_harmonic * frequency_hz + deviation_hz;
const auto refined_frequency = refined_probe_frequency / chosen_harmonic;
```

core/src/hapt.cpp:460–466

Prob, temel frekansta değil, **1., 3. veya 5. harmonikten en güçlüsünde** atılır — orası fazın en temiz okunduğu yerdir. Sonuç katına bölünerek temel frekansa geri indirilir; harmonikte ölçmek sapmayı da k katı büyüttüğü için doğruluk artar.

Üç güvenlik kilidi

Kilit	Gerekçe
Periyodiklik $\geq 0,75$	Fazı okumaya degecek kadar temiz bir ton olmalı.
İki alt pencere arası genlik farkı ≤ 6 dB	Atak/geçiş karesinde enerji hızla değişir; faz kayması artık frekansı değil zarfı ölçer.
Düzeltilme ≤ 40 sent	Daha büyük bir "düzeltme" muhtemelen faz sarması (phase wrap) belirsizliğidir, gerçek rafine değil.

core/src/hapt.cpp:448–470 · eşikler: hapt.hpp:42–44

Ayrıca rafine yalnız **tam ve kurtarılmamış** bir pencerede denenir; kurtarma alt penceresinden gelen bir tahmin için hiç çalıştırılmaz.

5.4 Atak ve sönüm kurtarması

Tam pencere yeterince güvenli bir şey bulamadıysa, sebep pencerenin *içinde* notanın başlıyor veya bitiyor olması olabilir — periyodiklik tüm pencereye yayıldığı için sulanmıştır. HAPT iki kurtarma denemesi yapar:

1. **Atak varsayımı:** yalnız pencerenin **son** 768 örneğini yeniden analiz et (sinyal yeni periyodikleştirdi).
2. **Sönüm varsayımı:** yalnız pencerenin **ilk** 768 örneğini analiz et (sinyal periyodikti, şimdi ölüyor).

Başarılı bir kurtarma, kanıtın yalnız bir kısmını gördüğü için puanı %15 kırılır.

core/src/hapt.cpp:517–562

5.5 İki eşikli histerezis

HAPT'ın "eksik seslendirilmiş kare" sorununa (diğer üç motorun ortak zaafı) cevabı budur: **tek bir eşik yerine iki eşik**.

izlenen kontura ≤ 180 sent uzaklıkta $\rightarrow$ eşik 0,32 (*sustain*)
taze veya uzak bir aday $\rightarrow$ eşik 0,55 (*onset*)

```
double onset_periodicity_threshold{0.55};  
double sustain_periodicity_threshold{0.32};  
double sustain_max_cents_from_contour{180.0};
```

core/include/klarivision/core/hapt.hpp:50–52

Yani sürmekte olan bir nota, zayıflasa bile takip edilmeye devam eder; yeni ve farklı bir perde ise çok daha yüksek kanıt ister. Süreklilik burada da kanıt sayılır — V2'nin "stable weak contour" kuralının HAPT'taki karşılığı.

5.6 HAPTTracker: bırakış tespiti ve sıçrama muhafızı

Sustain eşiğinin kasten gevşek tutulmasının bir bedeli var: sönen bir reverb kuyruğu da o eşiği geçebilir. Bu yüzden HAPT'ın bırakış tespiti diğerlerinden bir adım daha ileridir ve **iki** tetikleyicisi vardır:

Tetikleyici	Yakaladığı durum
2 ardışık %20 keskin düşüş	Sıradan nota bitişi (release)
RMS, son tepesinin %35'inin altına indi	Yumuşakça sönen oda reverb kuyruğu

Kurtulma koşulu: zarfın son tepenin %40'ına geri tırmanması.

```
if (release_falls_ >= 2 || decayed_below_recent_peak) release_active_ = true;  
if (release_active_ && rms >= release_rms_peak_ * config_.release_recovery_ratio) {  
    release_active_ = false; release_falls_ = 0;  
}
```

core/src/hapt.cpp:653–659

Kritik sıra: bırakış tespiti **tahminden önce ve ondan bağımsız** çalışır. Yani hâlâ sönmekte olan bir kuyruk, tahmincinin "yeterince gür ve periyodik" dediği bir karede bile bastırılır.

Aşağı harmonik sıçraması muhafızı VPM-like'inkiyle aynı desendir; farkları: yalnız 1/3 ve 1/2 oranlarına bakar (2/3 yok), oran toleransı 30 senttir (110 değil) ve 2 onay ister (3 değil).

core/src/hapt.cpp:52–66 ve 683–708

5.7 Bir karenin tam yolculuğu

1536 örnek

↓ DC çıkar + RMS kapısı	hapt.cpp:492
↓ NSDF tablosu (kenar dolgulu)	hapt.cpp:76
↓ yerel maksimumlar $\geq 0,05 \rightarrow$ hipotez listesi	hapt.cpp:172
↓ 1. geçiş: yarım/üçte-bir veto + tek doluluk + enerji	hapt.cpp:190
↓ 2. geçiş: anlamlılık ağırlığı + nihai puan	hapt.cpp:327
↓ atak / sönüm kurtarması (768 örneklik alt pencere)	hapt.cpp:517
↓ iki eşikli histerezis (0,55 onset / 0,32 sustain)	hapt.cpp:564
↓ faz kilitli anlık frekans rafinesi (≤ 40 sent)	hapt.cpp:404
↓ HAPTTracker: bırakış tespiti + sıçrama muhafızı	hapt.cpp:637
↓ boşluk köprüleme (yin_v1 ile ortak; taban 0,60/0,30)	analysis_engine.cpp:274

EngineFrame{ zaman, frekans, güven }

HAPT'ın gevşetilmiş köprü ucu

Ortak `append_bridged` çağrısında yalnız HAPT için **asimetrik** bir eşik verilir: güçlü uç 0,60'ı geçmeli ama zayıf uç için taban 0,30'a indirilir. Gerekçe kodda yazılı: bir kopmanın çıkış kenarı, tam da motorun kendi güveninin kesinti tarafından en çok bozulduğu yerdir; iki ucun da aynı barajı geçmesini istemek gerçek kısa boşlukları eksik köprüler.

core/src/analysis_engine.cpp:281–288, 933–934

6. pYIN — çevrimdışı referans

*pYIN, uygulamada kullanıcıya sunulan dört motordan biri değildir; **çevrimdışı referans eğrisi** olarak kullanılır. Kendi uygulamamız değil, librosa'nın uygulamasıdır — ama neden bu kadar iyi olduğunu bilmek, dört motorun tasarım kararlarını anlamak için gereklidir.*

src/klarivision/pitch/pyin.py – librosa.pyin adaptörü

6.1 YIN'den farkı: tek eşik yerine olasılık dağılımı

Klasik YIN'de tek bir kritik karar vardır: "CMND eğrisinde 0,15 eşikinin altına düşen ilk dip". Bu eşik **keyfidir** ve tek bir yanlış seçim tüm kareyi mahveder.

pYIN'in birinci fikri: **eşiği seçme, hepsini dene**. Farklı eşik değerleri üzerinde bir olasılık dağılımı (beta dağılımı) tanımlanır ve her eşik değerinin verdiği aday, o eşik olasılığıyla ağırlıklandırılır. Sonuç, tek bir frekans değil, kare başına **ağırlıklı bir aday kümesidir**.

6.2 İkinci ve asıl fikir: tüm kayıt üzerinde Viterbi

Aday kümeleri sonra bir **gizli Markov modeline** (HMM) verilir ve tüm kayıt boyunca en olası yol çözülür.

Neden pYIN oktav hatası yapmıyor

Yalıtılmış bir 1200 sentlik sapma, **küresel** bir yol arayışında geçiş maliyetini asla geri ödeyemez. Bir kare "oktav aşağı" demek için, o kareye girmenin ve o kareden çıkmanın maliyetini birlikte ödemek zorundadır. Nedensel bir motor bunu göremez, çünkü çıkış maliyetini henüz bilmez.

Ölçüm bunu doğruluyor: Şükrü Tunar hakemlik kümesinde işaretlenen 47 oktav hatasının **33'ü yalnız 1–3 kare sürüyor** (medyan 2, en uzun 9) ve doğru izlemenin tam ortasında oturuyor. pYIN bu tür hataları yapmıyor; çünkü kararı tek karede vermiyor.

Bu gözlem, Bölüm 7'deki çevrimdışı iyileştirme katmanının doğrudan gerekçesidir.

6.3 Kullanım biçimi ve maliyeti

```
frequencies, voiced, probability = librosa.pyin(
    samples[analysis_start:analysis_end],
    fmin=settings.minimum_frequency_hz, fmax=settings.maximum_frequency_hz,
    sr=sample_rate, frame_length=settings.frame_length, hop_length=settings.hop_length,
    resolution=settings.resolution_cents / 100, center=False, fill_na=np.nan,
)
```

src/klarivision/pitch/pyin.py:67–77

Ayar	Değer	Not
frame_length	2048	@22,05 kHz = 93 ms
hop_length	110	5 ms — üretim motorlarından ~2 kat sık
frekans aralığı	110–2000 Hz	üretim motorlarından geniş
resolution	5 sent	HMM durum ızgarasının çözünürlüğü

Kayıt 3 saniyelik parçalara bölünerek işlenir; her parça bir çerçeve boyu geriden başlar (**overlap**) ve yalnız çekirdek bölgesi tutulur — böylece parça sınırlarında kenar etkisi oluşmaz.

src/klarivision/pitch/pyin.py:64–86

Neden pYIN canlı modda kullanılmıyor

Çünkü küresel Viterbi **tüm kaydı görmeyi** gerektirir. Bu, canlı çalarken tanıma gereği imkânsızdır. Dört üretim motorunun tamamının nedensel (causal) olmasının sebebi budur; V2'nin 5 kareli sabit gecikmesi, bu duvara verilen en yakın uzlaşmadır.

7. Çevrimdışı iyileştirme katmanı

`PitchEngineProfile::offline_track` uzun süre yer tutucuydu. Dün doldu: Çalışma (Study) modunda, nedensel izin üzerinde **geleceğe bakan** iki temizlik pası çalışıyor. Canlı yol hiç değişmedi — `analyse_causal` ve her `ProductionPitchSession` gecikme sözleşmesini koruyor.

7.1 Neden gerekti

Bölüm 6.2'deki ölçüm: işaretli 47 oktav hatasının 33'ü yalnız 1–3 kare sürüyor. Nedensel bir motor bunları göremez, çünkü hatanın ne kadar sürdüğünü ancak sonrasını görünce anlarsınız. pYIN görebiliyor. Öyleyse çevrimdışı analizde biz de görebiliriz.

7.2 Harmonik yol iyileştirmesi

Yayınlanmış her kare için, o karenin kendi analiz penceresi bulunur ve **harmonik ailesi** ölçülür: yayınlanan frekans ve onun $\frac{1}{3}x$, $\frac{1}{2}x$, $2x$, $3x$ akrabaları.

Kritik tasarım kararı: spektral kanıt fiyattır, kapı değil

Alternatifleri spektral desteğe göre **elemek** yanlıştı. Onarıma gerçekten muhtaç karelerde doğru temel frekans, ailenin en güçlü üyesinin çoğu zaman %2'sinin altında ölçülüyor — nedensel motorun o kareyi zaten bu yüzden yanlış bilmesinin sebebi tam olarak budur. Dolayısıyla her harmonik alternatif **erişilebilir kalır**; spektral desteği olmayan sadece *daha pahalıya* gelir.

$$\text{destek} = E_{\text{üye}} / E_{\text{ailenin en güçlüsü}}$$
$$\text{alternatif_puanı} = \text{güven} \cdot 0,72 \cdot [0,30 + 0,70 \cdot \text{clip}(\text{destek}, 0, 1)]$$

nedensel kararın kendisi her zaman 1,00 katsayısıyla yarışır

0,72 iskontosu şunu garantiler: bir kare **yalnız yol sürekliliği** talep ettiğinde yer değiştirir, yerel kanıt talep ettiğinde değil.

```
const double backing = kOfflineUnsupportedFloor +  
    (1.0 - kOfflineUnsupportedFloor) * std::clamp(support, 0.0, 1.0);  
const double scaled = confidence * kOfflineAlternativeDiscount * backing;  
options.push_back({family[member].frequency, scaled, std::log(std::max(scaled, 1e-6))});
```

core/src/analysis_engine.cpp:1123–1126 · sabitler: analysis_engine.cpp:992–1001

Geçiş maliyeti neden karesel?

$$\text{geçiş}(a \rightarrow b) = -\min((\text{cents}(a,b) / w)^2, 12)$$

$w = 150 \text{ sent (nedensel motorlar)} \cdot 250 \text{ sent (V2)}$

Karesel olması kasıtlıdır: tek bir 1200 sentlik sapma, gerçek bir gezinin çok sayıda küçük adımından **çok daha fazlasını** öder. Üst sınır (12) ise tek bir çılgin karenin tüm yol puanını domine etmesini engeller.

V2'ye daha gevşek genişlik verilmesinin sebebi Bölüm 3.5: zaten kendi beş kareli Viterbi'si var, iki kez sıkılaştırmak gerçek hızlı geçkileri bastırıyor.

```
double offline_transition_width_cents(const PitchEngineId id) {  
    return id == PitchEngineId::pitch_engine_v2 ? 250.0 : 150.0;  
}
```

core/src/analysis_engine.cpp:987–989

30 ms'den uzun aralıklı kareler ayrı koşular sayılır: bir artikülasyon boşluğu üzerinden süreklilik varsayılmaz. Seslendirme kararı (voiced/unvoiced) bu pasta **hiç** yeniden ele alınmaz; yalnız frekanslar oynatılır.

7.3 Başboş nokta temizliği

İkinci pas daha basit bir soruyu sorar: "bu kısa koşu gerçekten bir nota mı, yoksa sessizliğin ortasında kalmış bir kaza mı?"

Koşul	Değer
Koşu uzunluğu	≤ 7 kare
Her iki yanında sessizlik	≥ 40 ms
Koşunun en yüksek güveni	$< 0,80$

Üçü birden sağlanırsa koşu atılır.

İki yandan yalıtılmış olma koşulu neden var?

Gerçek süslemeleri korumak için. Bir *çarpma*, süslediği notaya bitişiktir — sessizliğin ortasında yalnız kalmaz. Dolayısıyla bu testi hiçbir zaman geçemez.

0,80 eşiği uydurulmuş değil: dinleyicinin işaretlediği her başıboş noktada **doğru davranıp sessiz kalan motor VPM-like'tı**, ve diğerlerinin yayınladığı koşular 0,66–0,81 güven aralığında tepe yapıyordu. Kısa ve yalıtılmış bir koşuyu VPM-like'ın kendi yayın barajına tutmak kuraldır; o karelere uydurulmuş bir sayı değil.

core/src/analysis_engine.cpp:1009–1020 (gerekçeli sabitler) · 1023–1058 (uygulama)

7.4 Nerede devreye giriyor

```
std::vector<EngineFrame> PitchEngine::analyse(const std::span<const float> samples, const double rate) {
    auto baseline = analyse_causal(samples, rate);
    if (profile_ != PitchEngineProfile::offline_track || baseline.empty()) return baseline;

    return drop_offline_stray_runs(
        refine_offline_harmonics(baseline, samples, rate, config_, id_)
    );
}
```

core/src/analysis_engine.cpp:1202–1212

Yani: **Çalma Modu** (canlı mikrofon) ham nedensel izi görür; **Dinleme/Çalışma Modu** (dosya analizi) bu iki pastan geçmiş hâlini görür. İki mod arasındaki fark artık yalnız gecikme değil, kanıt miktarıdır.

8. Karşılaştırma

Dördü de aynı soruyu soruyor ama farklı kanıtlara bakıyor. Bu bölüm, her birinin neyi nasıl yaptığını yan yana koyuyor — ve elimizdeki tek gerçek ölçümü.

8.1 Yapı taşları

	YIN v1	Pitch Engine V2	VPM-like	HAPT v1
Çekirdek fonksiyon	CMND (fark)	YIN + ACF + NSDF + spektral	Normalize ACF	NSDF
Aday sayısı	≤ 12	Dört kaynaktan, hepsi	1 (seçilmiş tepe)	Tüm NSDF tepeleri
Spektral kanıt	Tek frekans probu (oktav kurtarma + hayalet)	SWIPE' çekirdeği (tam FFT)	Akraba testi (1/3, 1/2, 1, 2, 3x)	Yarım/üçte-bir ızgara + tek doluluk
Faz kullanımı	—	—	—	Evet (IF kilidi)
Karar biçimi	Anlık, süreklilik cezalı	5 kare Viterbi	Anlık + veto/histerezis	Anlık, iki eşikli
Gecikme	0 kare	5 kare (≈53 ms)	0 kare	0 kare
Yayın barajı	0,55 (taze: 0,76)	0,70 / 0,55	0,80	0,55 / 0,32
Aşağı sıçrama onayı	3 kare (>900 sent)	harmonic_filter	3 kare (+2,5x vetosu)	2 kare
Hesap yükü	Düşük	Yüksek	Orta	Orta-yüksek

8.2 Aynı sorunlara verilen farklı cevaplar

"Bu aday gerçekten temel frekans mı, yoksa hayalet mi?"

YIN v1	2x ve 3x katlarının genişliğini kendi genişliğiyle kıyasla; oran 6'yı geçiyorsa kademeli ceza. Oktav terfisi için ayrıca teke harmonik doluluğu ara.
V2	Dört bağımsız tahminci ne diyor? İkisi 55 sent içinde anlaşıyorsa +0,20. Ayrıca SWIPE' asal harmonik çekirdeği.
VPM-like	Gerçek spektrumda 1/3, 1/2, 1, 2, 3x akrabalarını ölç; yerel tepe + mutlak + görel testleri geçeren pes olanı seç. Ama asla yukarı terfi ettirme.
HAPT	"Bir kat fazla pes olsaydım enerji nerede olurdu?" diye sor. Yarım/üçte-bir ızgarayı ölç. Pencere çözemiyorsa NSDF katlarına düş.

"Nota bitti mi, yoksa sadece kısıldı mı?"

YIN v1	2 ardışık %20 düşüş → bırakış. İki ayrı RMS tepesi (hızlı/yavaş sönen).
V2	Aynı desen, ama durum kare başına kuyruklanır — 5 kare geciken karar asla başka bir pencerenin kapısını uygulamaz.
VPM-like	Tam durum makinesi: release_suspected → aynı kontura mı döndü, farklı kontura mı? Ayrıca waiting_for_attack .
HAPT	İki tetikleyici: 2 keskin düşüş veya tepenin %35'inin altına inme (reverb kuyruğu). Tahminden bağımsız çalışır.

"Kısa bir kopma oldu, ne yapmalı?"

YIN v1 ve HAPT ortak [append_bridged](#) 'i kullanır (≤7 kare, iki uç ≤90 sent). V2 ve VPM-like kendi köprüleme mantıklarına sahiptir; V2 ayrıca **başlangıç köprüsü** ile notanın gerçek atak zamanını geriye dönük kurtarır.

8.3 Ölçülmüş sonuçlar

Ölçümün kaynağı

Şükrü Tunar uşşak taksemi, **85 aralık**, 8,7 saniye. Yer gerçeği bir **dinleyicinin hükmüdür** — pYIN değil, motorlar arası medyan değil. Kayıt: `data/annotations/sukru-tunar-ussak-taksim.verdicts.v1.json`. Aşağıdaki sayılar `offline_track` profiline aittir.

Motor	Etiketli kusur	Tur başı	Tur sonu	Oktav (başlangıç)	Oktav (şimdi)
YIN v1	38	35	23	14	2
Pitch Engine V2	24	24	22	3	1
VPM-benzeri	46	46	33	18	5
Harmonik-Faz (HAPT)	50	45	34	12	1
Toplam oktav hatası: 47 → 9 · hiçbir motor kötüleşmedi					

docs/TEST_BASELINE.md:85–98

Okunacak birkaç şey var:

- **V2 zaten en az oktav hatası yapan motordur (3)**. Viterbi'si olan tek motor o; Bölüm 6.2'nin tezini doğrudan doğruluyor.
- **YIN v1'in 14 → 2 düşüşü** büyük ölçüde tek-harmonik doluluk koşulundan geliyor: işaretli 13 oktav hatasının 12'si oktav kurtarma pasından kaynaklanıyordu.
- **VPM-like'ın 18 → 5 düşüşü** büyük ölçüde 120 Hz alt sınırından geliyor (lag arama aralığının daralması).
- Oktav **dışı** kusurlar (boşluk, kaçık nokta, zamanlama) hâlâ önemli sayıda — bu ölçüm oktav sorununa odaklıydı.

Bu tablo bir sıralama değildir

Tek bir kaydın 85 aralığı üzerinde, tek bir dinleyicinin hükümlerine göre yapılmış bir ölçüm. Farklı bir kayıta (`gercek-klarnet-calm`) çevrimdışı iyileştirme dört motorda da tek kare bile değiştirmede. Uygulamadaki motor seçimi de bir kalite sıralaması değildir; ilk açılıştaki YIN v1 seçimi yalnız geriye uyumluluk içindir.

8.4 Hangi motor neyi iyi yapıyor

Motor	Güçlü yanı	Bedeli
YIN v1	Basit, hızlı, öngörülebilir. Okunabilir referans uygulaması var. Sıfır gecikme.	Tek karelik kararlar; kanıtı erken çökertiyor.
Pitch Engine V2	Kanıt çeşitliliği + yol araması. Tek karelik hatalar yola oturmuyor. 900 Hz üstünde disiplinli.	≈53 ms gecikme, kare başına belirgin hesap (dört tahminci + FFT).
VPM-like	Sessiz kalması gerektiğinde sessiz kalıyor . En yüksek yayın barajı (0,80), en ayrıntılı atak/bırakış durum makinesi. Katlardan periyot rafinesi tiz seslerde kesin.	Muhafazakârlık boşluk kusuru olarak geri dönüyor; en çok etiketli kusur onda.
HAPT v1	Klarnet fiziğini doğrudan karar kuralına çeviriyor. Çözünürlük farkındalığı. Faz kilidiyle alt-sent doğruluk. İki eşikli histerezis "eksik kare" sorununu azaltıyor.	Gevşek sustain eşiği, sönen kuyrukları yakalamak için ayrı bir bırakış tespiti gerektiriyor.

Ek A — Parametreler

A.1 Ortak üretim yapılandırması

core/include/klarivision/core/analysis_engine.hpp:15–37

Alan	Değer	Anlamı
minimum_frequency_hz	120,0	En pes hipotez. $\tau_{\max}$ 'ı kısıtladığı için doğrudan bir doğruluk parametresidir.
maximum_frequency_hz	1500,0	En tiz hipotez $\rightarrow \tau_{\min}$
minimum_rms	0,015	Sessizlik kapısı (tüm motorlar için ortak)
window_size	1536	34,8 ms @44,1 kHz
hop_size	512	11,6 ms $\rightarrow \sim 86$ ölçüm/sn

A.2 Motora özel eşikler

Eşik	Değer	Nerede
YIN v1		
CMND aday eşiği	0,50	analysis_engine.cpp:122
Taze aday barajı	0,76	analysis_engine.cpp:240
Aday değerlendirme tabanı	0,55	analysis_engine.cpp:249
Süreklilik cezası	0,30 / 700 sent	analysis_engine.cpp:253
Hayalet oran eşiği	6,0	harmonic_arbitration.cpp:74
Oktav kurtarma baskınlığı	8×, >800 Hz	analysis_engine.cpp:159–171
Aşağı sıçrama	>900 sent, 3 kare	analysis_engine.cpp:558–573
Yayın/köprü tabanı	0,55	analysis_engine.cpp:933
Pitch Engine V2		
YIN adayı eleme	0,50	v2_session.cpp:313
Uzlaşma penceresi	55 sent	v2_session.cpp:371
Emisyon ağırlıkları	0,40 / 0,20 / 0,40	v2_session.cpp:379
Spektral aday tabanı	0,90	v2_session.cpp:387
Viterbi gecikmesi	5 kare	analysis_engine.cpp:420 (kurucu)
Geçiş genişliği	700 sent	fixed_lag_tracker.hpp:35
Oktav ek ücreti	+0,12 (≥ 850 sent)	fixed_lag_tracker.cpp:30
Yayın barajı	0,70 / 0,55	v2_session.cpp:490–495
VPM-like		
En az periyodiklik	0,38	vpm_like.hpp:52
Çıkış güveni	0,80	vpm_like.hpp:53
"En güçlüye yakın" oranı	0,90	vpm_like.hpp:54
Görelî spektral destek	0,080	vpm_like.hpp:55
Mutlak spektral destek	0,005	vpm_like.hpp:56
En yüksek periyot katı	6	vpm_like.hpp:57
Sıçrama tanımı	≥ 650 sent, ± 110 sent	vpm_like.cpp:23–33
Üst hat vetosu	2,5×	vpm_like.cpp:124

HAPT v1		
Harmonik sayısı	5	hapt.hpp:38
Tek harmonik eşiği	0,10	hapt.hpp:39
IF kilidi periyodikliği	0,75	hapt.hpp:42
IF kilidi düzeltme sınırı	40 sent	hapt.hpp:43
IF kilidi genlik sınırı	6 dB	hapt.hpp:44
Onset / sustain eşiği	0,55 / 0,32	hapt.hpp:50–51
Sustain kontur mesafesi	180 sent	hapt.hpp:52
Sıçrama toleransı / onay	30 sent / 2 kare	hapt.hpp:55–56
Bırakış oranları	0,80 / 0,35 / 0,40	hapt.hpp:67–69
Çevrimdışı katman		
Desteksiz alternatif tabanı	0,30	analysis_engine.cpp:997
Alternatif iskontosu	0,72	analysis_engine.cpp:1000
Bölüm boşluğu	30 ms	analysis_engine.cpp:1003
En yüksek geçiş cezası	12,0	analysis_engine.cpp:1007
Geçiş genişliği	150 / 250 sent	analysis_engine.cpp:987–989
Başıboş koşu	≤7 kare, 40 ms, 0,80	analysis_engine.cpp:1011–1020

Ek B — Dosya haritası

Dosya	İçeriği
Ortak / oturum	
<code>core/src/analysis_engine.cpp</code>	Üretim oturumu. YIN v1'in tamamı, VPM ve HAPT'ın oturum katmanı, köprüleme, çevrimdışı iyileştirme.
<code>core/include/.../analysis_engine.hpp</code>	<code>PitchEngineConfig</code> , <code>EngineFrame</code> , motor kimlikleri.
<code>core/src/harmonic_probe.cpp</code>	Hann pencereli tek frekans DFT probu; ana lob genişliği.
<code>core/src/harmonic_arbitration.cpp</code>	<code>spectral_existence</code> — hayalet alt-harmonik tespiti.
YIN v1	
<code>src/klarivision/pitch/yin.py</code>	Okunabilir referans uygulaması (108 satır).
<code>core/src/analysis_engine.cpp:86–176</code>	<code>yin_candidates</code> — CMND + oktav kurtarma.
<code>core/src/analysis_engine.cpp:194–272</code>	<code>causal_yin_choice</code> — hayalet + süreklilik puanlaması.
<code>macos/.../LivePitchAnalyzer.swift:3772</code>	vDSP hızlandırmalı canlı kopya.
Pitch Engine V2	
<code>core/src/pitch_engine_v2_session.cpp</code>	Aday üretimi (4 kaynak), puanlama, yayın kapıları.
<code>core/src/fixed_lag_tracker.cpp</code>	Sabit gecikmeli Viterbi.
<code>core/src/swipe_prime.cpp</code>	FFT + SWIPE' asal harmonik çekirdeği.
<code>core/src/mpm.cpp</code>	NSDF / McLeod aday üretici (referans sürüm).
<code>core/src/pitch_engine_v2.cpp</code>	Sade seçim sınırı (<code>select_candidate</code>).
VPM-like	
<code>core/src/vpm_like.cpp</code>	ACF, kat rafinesi, spektral düzeltme, <code>VPMLikeTracker</code> .
<code>core/include/.../vpm_like.hpp</code>	Yapılandırma ve tanı yapıları.
<code>docs/VPMLikePitchEngine.md</code>	Yöntemin türetimi ve kaynakları.
HAPT v1	
<code>core/src/hapt.cpp</code>	NSDF, harmonikler-arası veto, IF kilidi, <code>HAPTTracker</code> .
<code>core/include/.../hapt.hpp</code>	Yapılandırma; her eşğin gerekçesi yorumda.
<code>docs/HAPTPitchEngine.md</code>	Tam türetim, $\pm 93,75$ Hz aralığı, bilinen sınırlar.
pYIN ve ölçüm	
<code>src/klarivision/pitch/pyin.py</code>	librosa.pyin adaptörü (çevrimdışı referans).
<code>scripts/quick_pitch_check.py</code>	Motorları dinleyici hükümlerine karşı ~14 s'de puanlar.
<code>scripts/pitch_engine_divergence_map.py</code>	Motorların ayrıştığı yerleri bulur, inceleme sayfasını üretir.
<code>docs/TEST_BASELINE.md</code>	Ölçüm kayıtları ve gerekçeler.

Bu belge `37585a4` kod sürümüne göre yazıldı. Satır numaraları o commit'e aittir; kod değişikçe kaymaları beklenir — her alıntının yanındaki fonksiyon/sabit adı kalıcı çapadır.